Ontwikkeling autonome tap

Timo Nieuwenhuijs
694995

Hogeschool Inholland
B Technische Informatica VT

Project Domotica
N. Ottens
M. Oldenburg

26 maart 2026

Inhoudsopgave

1 Inleiding **3**

2 Probleemstelling **4**
2.1 Probleemanalyse . . . 4
2.2 Vooronderzoek . . . 4
Eisen . . . 4
Mogelijke meetmethoden . . . 4
Keuze van de sensor . . . 4
Keuze van de actuator . . . 5
Systeemoverzicht . . . 5
2.3 Vraagstelling . . . 5

3 Methode **6**
3.1 Dataverwerking . . . 6
3.2 Logica . . . 7
3.3 Vormgeving . . . 8
3.4 Testvoorwaarden . . . 9

4 Resultaten **10**
4.1 Dataverwerking . . . 10
4.2 Logica . . . 10
4.3 Vormgeving . . . 10
4.4 Testvoorwaarden . . . 11

5 Conclusie **12**
5.1 Dataverwerking . . . 12
5.2 Logica . . . 12
5.3 Vormgeving . . . 12
5.4 Testvoorwaarden . . . 12
5.5 Hoofdvraag . . . 12

6 Aanbevelingen **13**

7 Discussie **13**

Referenties **14**

Bijlagen **15**

A Dataverwerking **15**
A.1 ERD . . . 15
A.2 Dashboard . . . 16

B Flowcharts **17**
B.1 Raspberry Pi . . . 17
B.2 Microcontroller . . . 18

C Schema's **19**
C.1 Elektrisch schema 19
C.2 Soldeer schema 20

D Broncode **21**
D.1 Microcontroller 21
D.2 Raspberry Pi 5 26

1 Inleiding

De groei van smart home technologieën is de afgelopen jaren sterk toegenomen[1]. Denk aan thermostaten, blinderingen, lampen en wasmachines. Een slimme watertap is een van de apparaten die voor de consument op kleine schaal niet makkelijk toegankelijk is. Deze komen eerder veel voor bij fastfoodketens of werkplekken en zijn vaak groot en duur[2]. Alhoewel slimme kranen wel bestaan, werken deze niet compleet autonoom.

De opdrachtgever vereiste een systeem te ontwikkelen, dat gebruikt maakt van één sensor en één actuator. In combinatie met deze eis en het probleem hierboven is ervoor gekozen om een autonome (kleinschalige) watertap te maken. De tap bestaat uit twee delen, een reservoir om de beschikbare hoeveelheid vloeistof bij te kunnen houden en de kraan zelf, die autonoom een glas kan vullen en afwegen.

Het doel van het gehele project is het ontwikkelen van een slimme watertap. In dit onderzoek wordt gekeken naar de realisatie van het tap gedeelte. De hoofdvraag die hierbij aansluit luidt: Hoe komt de ontwikkeling van een autonome tap tot stand?

In de volgende secties, zal eerst het probleem verder worden uitgelegd en de verschillende eisen worden benoemd. Vervolgens wordt er gekeken hoe er aan deze eisen kan worden voldaan. Tenslotte wordt uit de resultaten een conclusie getrokken, waaruit een aantal aanbevelingen en verbeterpunten volgen.

2 Probleemstelling

2.1 Probleemanalyse

Het technische probleem betreft het real-time bepalen van de inhoud van een glas met een sensor en het daarop reageren met een actuator. De manier waarmee de inhoud wordt bijgehouden en de waterstroom wordt gecontroleerd spelen een belangrijke rol. De verzonden data moet worden opgenomen in een database en de data moet ook kunnen worden ingezien. Daarnaast moet de data ook real-time worden bijgehouden. Verder zijn er vanuit de opdrachtgever verschillende eisen gesteld, waaraan het systeem moet voldoen. Hetzelfde geldt voor een deel van de elektrische componenten. Tenslotte is het verplicht dat de microcontroller zelf geen beslissingen neemt, de microcontroller mag enkel data verzenden en aansturing-commando's ontvangen. Alle code moet geschreven worden in programmeertaal C[3].

2.2 Vooronderzoek

Eisen

Voor het ontwikkelen van een autonome tap zijn een aantal eisen opgesteld. Het systeem moet in staat zijn om een glas automatisch te vullen zonder tussenkomst van de gebruiker. Daarnaast moet het systeem stoppen met vullen zodra een vooraf bepaalde hoeveelheid water is bereikt. De gebruikte microcontroller moet een Challenger+ RP2350 WiFi6/BLE5[4] zijn, als database server moet een Raspberry Pi 5 (Pi)[5] worden gebruikt. Alle software nodig op de Pi, moet open-source zijn en kunnen draaien op een Linux distributie. Het vooraf bepaalde communicatie protocol tussen microcontroller en Pi is MQTT. Door deze eisen moet er tijdens het bepalen van een geschikte sensor en actuator rekening worden gehouden met de limieten van de microcontroller, evenals de keuze in software op de Pi.

Mogelijke meetmethoden

Om te bepalen wanneer een glas gevuld is, zijn verschillende meetmethoden mogelijk. Een eerste methode is het meten van tijd, waarbij de kraan gedurende een vaste tijd openstaat. Deze methode is echter onnauwkeurig, omdat de hoeveelheid doorgelaten water afhangt van de doorstroom. Een tweede methode is het gebruik van een flowsensor, waarmee de hoeveelheid doorgestroomd water direct kan worden gemeten. Hoewel dit een nauwkeurige methode is, brengt het extra complexiteit met zich mee in zowel hardware als software, software matig moet de gemeten doorstroom worden omgerekend naar een specifieke hoeveelheid water. Een derde methode is het meten van het gewicht van het glas. Hierbij wordt gebruikgemaakt van een weegschaal om het gewicht continu te monitoren tijdens het vullen.

Keuze van de sensor

Op basis van de vergelijking tussen de meetmethoden is gekozen voor het gebruik van een weegschaal. Deze methode biedt een goede balans tussen nauwkeurigheid en eenvoud. Water heeft een soortelijke massa van 0,9970 $\mathrm{g\,cm^{-3}}$[6, p.796], wat betekent dat het gewicht van het water direct gerelateerd is aan het volume. Hierdoor kan op basis van het gemeten gewicht worden bepaald hoeveel water zich in het glas bevindt, 1 mm is immers 1 g. Dit maakt het mogelijk om eenvoudig de huidige inhoud van het glas te monitoren. Er is gekozen voor een *Unit Mini Scales[7]*. Deze sensor is gekozen, omdat deze werkt op I2C logica, wat direct compatibel is met de microcontroller. Verder is de sensor al gebruiksklaar, wat integratie vereenvoudigt.

Keuze van de actuator

Voor het aansturen van de watertoevoer is gekozen voor een magneetventiel. Deze kan eenvoudig worden aangestuurd met een digitaal signaal en is in staat om de waterstroom snel te openen en te sluiten. Het gekozen magneetventiel[8] werkt op 5V, waardoor er geen externe voeding en relais nodig is. Tussen de microcontroller en het gekozen ventiel moet wel een H-brug worden geplaatst.

Systeemoverzicht

Het systeem bestaat uit een combinatie van een sensor, microcontroller, server en actuator. De weegschaal meet continu het gewicht van het glas en stuurt deze gegevens naar de microcontroller. Vervolgens worden de gegevens via het MQTT-protocol doorgestuurd naar de server, waar de logica wordt uitgevoerd. Op basis van deze logica wordt het magneetventiel aangestuurd om het glas te vullen of te stoppen.

2.3 Vraagstelling

Hoofdvraag: Hoe komt de ontwikkeling van een autonome tap tot stand?

Voor het beantwoorden hiervan zijn de volgende deelvragen opgesteld:

1. Op welke wijze wordt alle data verwerkt?
2. Met welke logica weet het systeem de staat van het glas?
3. Hoe kan de tap zo worden vormgegeven dat het glas tegelijkertijd gevuld en gewogen kan worden?
4. Hoe kunnen alle vooraf gestelde voorwaarden worden getest?

3 Methode

3.1 Dataverwerking

'Op welke wijze wordt alle data verwerkt?' Er is gekozen deze vraag te beantwoorden met een ontwerp. Hiermee kan duidelijk worden gemaakt hoe de Pi intern moet functioneren.

Er wordt een blokschema opgesteld, dat laat zien wat er allemaal nodig is om succesvol alle data te verwerken. De verwerking wordt met behulp van verschillende applicaties gerealiseerd. Waaronder een MQTT broker, een database server en een dashboard. Het blokschema geeft een overzicht van de benutte applicaties en hoe deze met elkaar functioneren.

Mosquitto[9] wordt als MQTT server gebruikt, voor het ontvangen van data en het versturen van aansturing-commando's. Mariadb[10] functioneert als database server, waar alle benodigde data wordt geregistreerd en met DBeaver[11] kan de database eenvoudig worden ingezien. Grafana[12] wordt gebruikt om alle data te visualiseren op het dashboard. Voor de interne communicatie tussen het MQTT protocol en de database wordt een C programma ontwikkeld. MariaDB en Mosquitto, beschikken beide over geïntegreerde besturingsmogelijkheden met C code.

In het opgestelde blokschema moeten alle gebruikte applicaties voorkomen. De onderlinge relaties en waarvoor deze dienen moeten ook worden vermeld.

3.2 Logica

Voor het beantwoorden van de vraag 'Met welke logica weet het systeem de staat van het glas?' wordt een ontwerp gemaakt. Hiermee kan duidelijk in kaart worden gebracht wat de gedachtegang is achter de werking van de tap.

Er worden twee flowcharts gemaakt waarin alle keuzes en voorwaarden, die de systemen moeten maken, verwerkt zijn. De ene flowchart laat zien hoe het programma op de Pi werkt, de ander de microcontroller. Op deze wijze is overzichtelijk te zien hoe het programma functioneert en hoe deze zich behoort te gedragen.

Aan beide flowcharts zijn verschillende eisen gesteld.
Eisen microcontroller:

- De kraan moet open en dicht kunnen op basis van MQTT-berichten.
- De sensor moet worden uitgelezen.
- De sensor data moet worden verzonden.
- De microcontroller moet een succesvolle wifi verbinding maken.

Eisen Pi:

- MQTT-berichten ontvangen en verzenden.
- Alleen de kraan openzetten als er een glas staat.
- De kraan sluiten als het vooraf bepaalde gewicht is bereikt.
- De kraan sluiten nadat het glas genoeg is gevuld.
- Het nulpunt van de weegschaal bepalen.
- Na het vullen de hoeveelheid naar de database loggen.

Beide programma's zijn geschreven in C. Het programma op de Pi is hetzelfde programma, wat ook de eerder genoemde interne communicatie laat functioneren [3.1].

3.3 Vormgeving

'Hoe kan de tap zo worden vormgegeven dat het glas tegelijkertijd gevuld en gewogen kan worden?'. Dit wordt gerealiseerd met een hardware-ontwerp. Zonder werkende tap is het onmogelijk een eindproduct op te leveren.

De tap wordt getekend in het CAD programma SolidWorks[13]. Bij het ontwerp wordt rekening gehouden met de plaatsing van de componenten en formaten van veelvoorkomende drinkglazen. Het eindontwerp wordt vervolgens gerealiseerd met een 3d-printer. Voor de electronica wordt eerst een elektrisch schema opgesteld in KiCad[14] en een visualisatie gemaakt in Fritzing[15]. Op basis van deze visualisatie wordt het perfboard gesoldeerd.
SolidWorks is het CAD programma wat tot beschikking is gesteld door de opdrachtgever, in combinatie met een 3d-printer kunnen er snel prototypes worden geproduceerd. Het elektrisch schema visualiseert hoe de elektrische componenten met elkaar verbonden zijn. Daarnaast helpt de tekening in Fritzing met voorkomen van soldeerfouten.

Aan het ontwerp zijn meerdere eisen gesteld:

- Er passen verschillende glazen onder.
- Componenten zijn correct aangesloten.
- Componenten en draden zijn degelijk gesoldeerd.
- Kans op waterschade is geminimaliseerd.
- Tap is gemakkelijk te assembleren.

3.4 Testvoorwaarden

'Hoe kunnen alle vooraf gestelde voorwaarden worden getest?'. Dit wordt gevalideerd met een aantal experimenten.

Eerst wordt op een breadboard getest of de solenoid succesvol kan worden aangestuurd, in plaats de solenoid zal een dc motor worden gebruikt. Vervolgens wordt er gekeken of de microcontroller succesvol het gewicht van de sensor kan aflezen. Daarna wordt er gekeken of de microcontroller de benodigde MQTT-commando's succesvol verstuurd en ontvangt. Bij het ontvangen wordt getest of de actuator activeert, hierbij wordt ook de richting van de solenoid getest. Dit kan bevestigd worden met lucht, water is nog niet nodig. Vervolgens worden de geprogrammeerde voorwaarden gecontroleerd, door gewicht op de sensor te plaatsen en te observeren hoe de motor hierop reageert. Als alles correct werkt, wordt de tap geassembleerd en nogmaals getest, ditmaal volgens operationeel gebruik.
De aansturing van de solenoid wordt eerst gesimuleerd met een dc-motor, omdat activering dan met blote oog waar te nemen is. Rotatie en richting van de motor kan overduidelijk worden waargenomen. Voordat er voorwaarden kunnen worden geprogrammeerd, moet de microcontroller eerst succesvol het gewicht kunnen uitlezen en versturen over MQTT naar de Pi. Voordat de solenoid kan worden getest met water, moet eerst worden bevestigd dat de controller succesvol 'open' en 'dicht' berichten kan ontvangen en de Pi deze commando's kan versturen. Het vooraf valideren van deze voorwaarden voorkomt een overstroming, indien deze niet meteen naar behoren functioneren.

Het experiment wordt als geslaagd beschouwd, wanneer de geassembleerde kraan alle bovengenoemde testen nogmaals met positief resultaat afrondt. Deze testen worden ook tijdens het ontwikkelproces getest. Indien de initiële test faalt wordt het programma aangepast en opnieuw getest, pas als het programma werkt naar behoren wordt er verder ontwikkeld. De resultaten worden opgenomen in een meetrapport. Het oningevulde meetrapport is opgenomen in tabel [1].

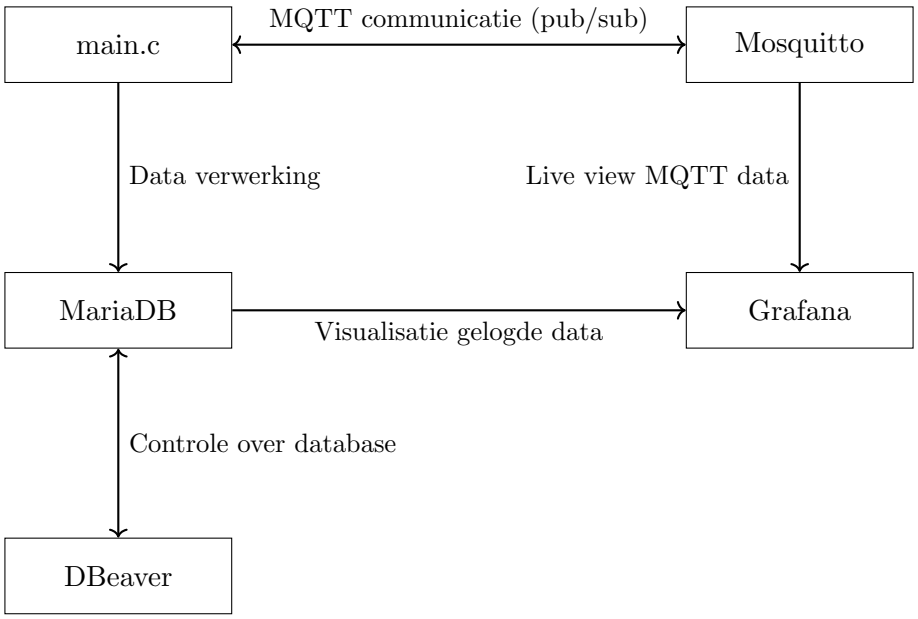
Tabel 1: Template voor testresultaten (leeg)

Testconditie	**Initieel**	**Definitief**	**Opmerking**
DC motor aansturen			
Sensor uitlezen			
MQTT versturen			
MQTT ontvangen			
Sensordata versturen			
Valve aansturen MQTT			
Richting valve correct			
Nulpunt opslaan			
Open voorwaarde			
Sluit voorwaarde			
Vroegtijdig sluiten			
Leeg herkenning			
Loggen database			

4 Resultaten

4.1 Dataverwerking

In figuur [1] is te zien hoe de verschillende applicaties met elkaar communiceren.



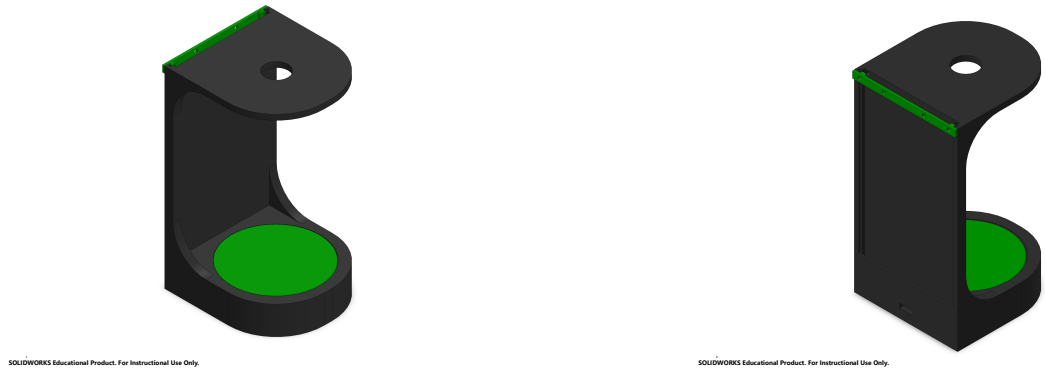
Figuur 1: Blokschema van de dataverwerking binnen het systeem

4.2 Logica

Om leesbaarheid te bevorderen zijn de volledige flowcharts opgenomen in de bijlage als figuur [5] en [6].

4.3 Vormgeving

Zowel het elektrisch schema, als Fritzing tekening zijn opgenomen in de bijlagen. Zie figuren [7] en [8]. De SolidWorks schets staat afgebeeld in figuur [2].



Figuur 2: Voor- en achterkant case

4.4 Testvoorwaarden

De testresultaten staan vermeld in tabel [2]. Waarbij 'N' staat voor gefaald en 'Y' voor geslaagd, wat er is aangepast aan de functie, om deze correct te laten werken, is terug te zien in de opmerkingen kolom.

Tabel 2: Meetrapport na uitvoering testen

Testconditie	**Initieel**	**Definitief**	**Opmerking**
DC motor aansturen	Y	Y	
Sensor uitlezen	N	Y	buffer-data naar andere variabel gekopieerd
MQTT versturen (Pi/Challenger)	N	Y	code herschreven door projectlid
MQTT ontvangen (Pi/Challenger)	N	Y	code herschreven door projectlid
Sensordata versturen	Y	Y	
Valve aansturen MQTT	Y	Y	
Richting valve correct	N	Y	draden op H-brug verwisseld
Nulpunt opslaan	N	Y	voorwaarde toegevoegd, wat het nulpunt alleen bij het eerste bericht registreert
Open voorwaarde	N	Y	1. stabiliteitscontrole toegevoegd 2. voorwaarde toegevoegd wat vroegtijdig opengaan vermijdt
Sluit voorwaarde	Y	Y	
Vroegtijdig sluiten	N	Y	marge aan startgewicht toegevoegd
Leeg herkenning	Y	Y	
Loggen database	Y	Y	

5 Conclusie

5.1 Dataverwerking

Het verwerken van de meetdata (gewicht) is gerealiseerd op de Pi. De microcontroller en Pi communiceren door middel van het MQTT-protocol. De gegevens worden opgeslagen in een database. Zowel de MQTT-berichten als database worden gevisualiseerd. Op deze wijze wordt de data verzameld, verwerkt en weergegeven. De vormgeving van de database is opgenomen in de bijlagen [fig. 3], evenals een overzicht van het dashboard [fig. 4].

5.2 Logica

De staat van het glas wordt bepaald op basis van het gewicht en vooraf gedefinieerde grenswaarden. Door het actuele gewicht te vergelijken met het leeggewicht en een startgewicht, kan het programma bepalen of er een glas aanwezig is, of deze gevuld wordt en wanneer die vol is. Er wordt gebruik gemaakt van een stabiliteitscontrole, om vroegtijdige opening en sluiting tegen te gaan. Deze logica maakt het mogelijk om betrouwbaar het glas te vullen zonder interferentie van de eindgebruiker.

5.3 Vormgeving

Het ontwerp resulteert in een constructie waarbij het glas direct op de weegschaal wordt geplaatst, het magneetventiel bevindt zich recht boven het glas. Hierdoor is gelijktijdig meten en vullen mogelijk. De afstand tussen het ventiel en het platform is 140mm, de diameter van het platform is 80mm. Hierdoor is het ontwerp geschikt voor glazen van verschillende maten. De scheidingswand tussen het perfboard en weegplatform biedt voldoende bescherming tegen opspattend water. Met behulp van de fritzing tekening zijn de componenten correct en degelijk gesoldeerd. Hiermee voldoet het ontwerp aan de gestelde eisen.

5.4 Testvoorwaarden

Aan de resultaten van de testen is te zien, dat het systeem voldoet aan de gestelde eisen. Verschillende functies werkten naar behoren bij de initiële test, zoals het versturen van de sensordata (nadat de MQTT-communicatie functioneerde). Een aantal functies zijn na aanpassingen bij de tweede, definitieve, test pas als werkend gevalideerd; waaronder de start voorwaarde en één van de sluit voorwaarden. Uit de definitieve resultaten kan worden geconcludeerd dat alle vereiste logica succesvol is geïntegreerd.

5.5 Hoofdvraag

Op basis van de voorafgaande conclusies kan worden geconcludeerd dat het mogelijk is om een autonome tap te ontwikkelen. Deze ontwikkeling is ontstaan door een combinatie van sensormetingen, server-side logica en aansturing van het magneetventiel via MQTT . De kraan beschikt over verschillende basisfuncties, waaronder het loggen van data en vult zonder invoer van de eindgebruiker het glas met een voorafbepaalde grenswaarde. Alle componenten zijn verwerkt in een passende behuizing, waardoor het eindproduct in gebruik kan worden genomen.

6 Aanbevelingen

Hoewel het huidige eindproduct functioneert en voldoet aan de eisen gesteld door de opdrachtgever, zijn er wel een aantal verbeterpunten:

- Via het dashboard zelf de hoeveelheid water kunnen bepalen.
- Via het dashboard de kraan handmatig aan kunnen sturen.
- Een herontwerp van de behuizing, waarbij een opvangbak wordt geïntegreerd.
- De tap laten communiceren met de actuatoren gebruikt in het reservoir (Ontwikkeld door mede-projectlid).

7 Discussie

Binnen het projectteam zijn verschillende aspecten door één projectlid ontwikkeld en gedeeld, waaronder de MQTT communicatie (R. Borst), de database communicatie en het dashboard (T. Nieuwenhuijs). Tijdens het onderzoek bleek het laten samenwerken van alle software en componenten lastiger te zijn dan verwacht, met name de communicatie en succesvolle aansturing van de actuator. Het onderzoek had verbeterd kunnen worden door testen van het systeem eerder uit te voeren, waardoor er de tijd zou zijn geweest uitgebreidere logica te implementeren. Desondanks toont het onderzoek aan dat een haalbaar resultaat kan worden gerealiseerd en de opgedane kennis na afronding van het project zeer waardevol is.

Referenties

[1] ABI Research, "APAC: smart home device shipments CAGR by country 2020-2026," July 2022.

[2] Tapclean, "Watertafeltaps van hoge kwaliteit | Tapclean."

[3] C, "C language."

[4] iLabs, "Challenger+ RP2350 WiFi6/BLE5."

[5] R. P. Ltd, "Buy a Raspberry Pi 5."

[6] D. R. Lide, *CRC Handbook of Chemistry and Physics: Internet Version 2005*. Boca Raton, FL: CRC Press, 2005.

[7] M5Stack, "Unit Mini Scales."

[8] tinytronics, "Magneetventiel - Bi-stabiel - 5V - Messing - G1/2"."

[9] R. A. Light, "Mosquitto: server and client implementation of the MQTT protocol," *Journal of Open Source Software*, vol. 2, p. 265, May 2017.

[10] MariaDB, "MariaDB Community Server: Free, Open Source Database."

[11] DBeaver, "DBeaver Community | Free Open-Source Database Management Tool."

[12] Grafana, "Grafana: The open and composable observability platform."

[13] Dassault Systèmes, "Solidworks," 2025.

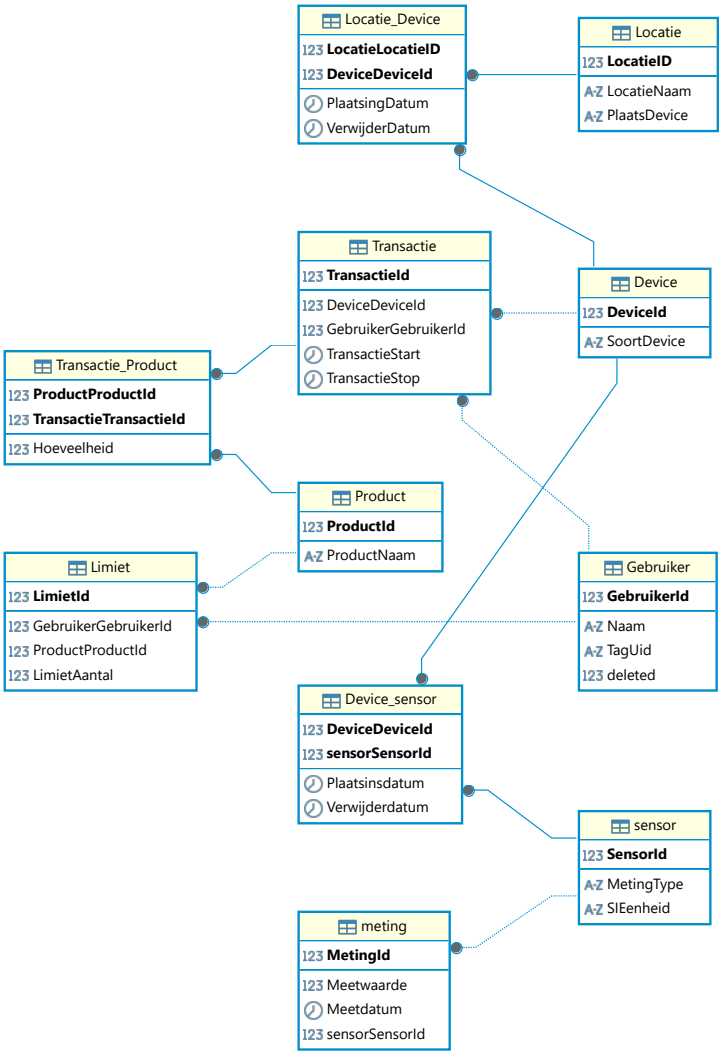
[14] KiCad Developers, "Kicad," 2026.

[15] Fritzing Foundation, "Fritzing," 2026.

Bijlagen

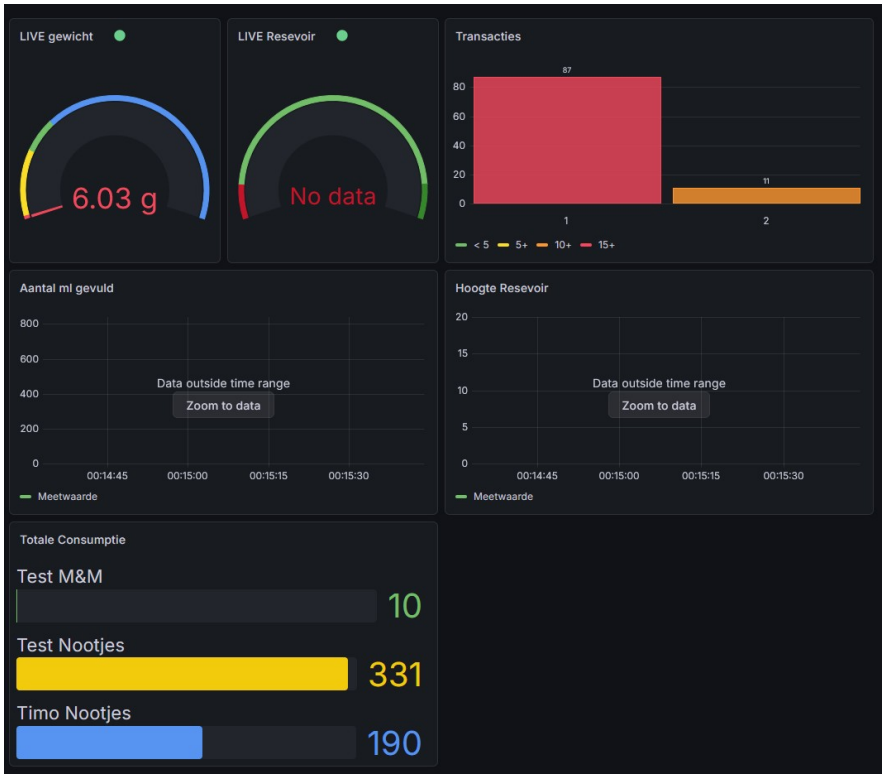
A Dataverwerking

A.1 ERD



Figuur 3: Entity-Relation Diagram database

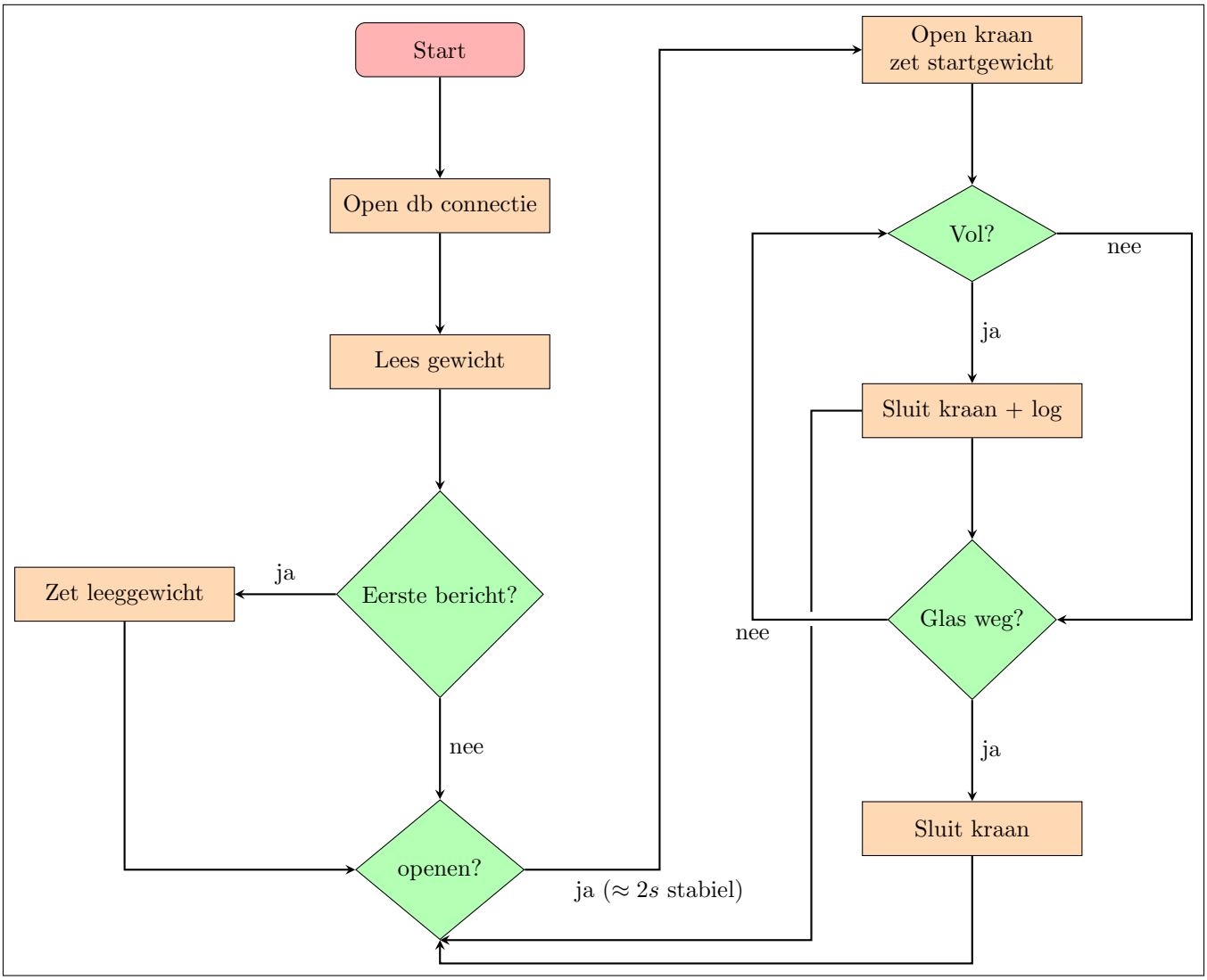
A.2 Dashboard



Figuur 4: Overzicht dashboard

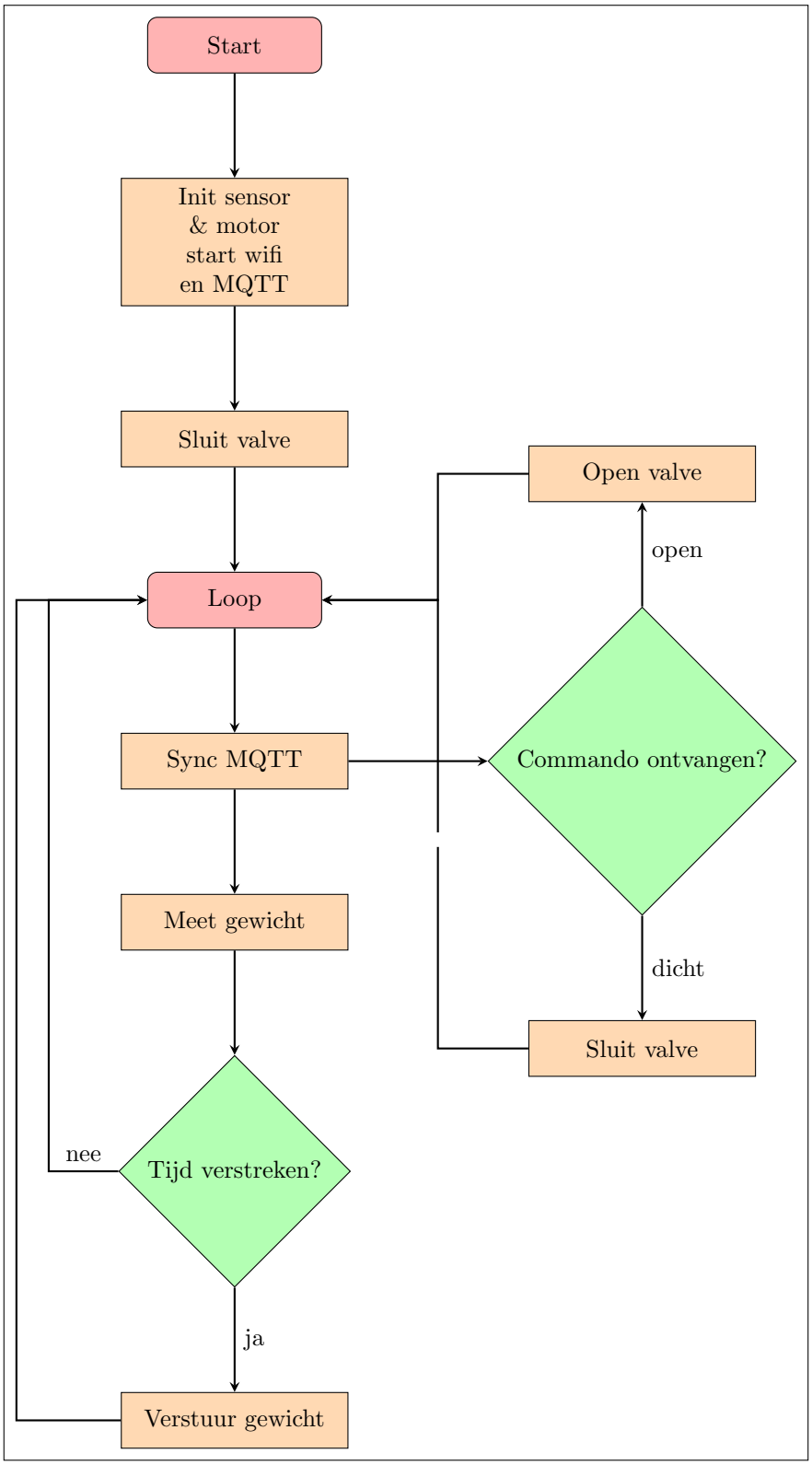
B Flowcharts

B.1 Raspberry Pi



Figuur 5: Flowchart van de Raspberry Pi logica

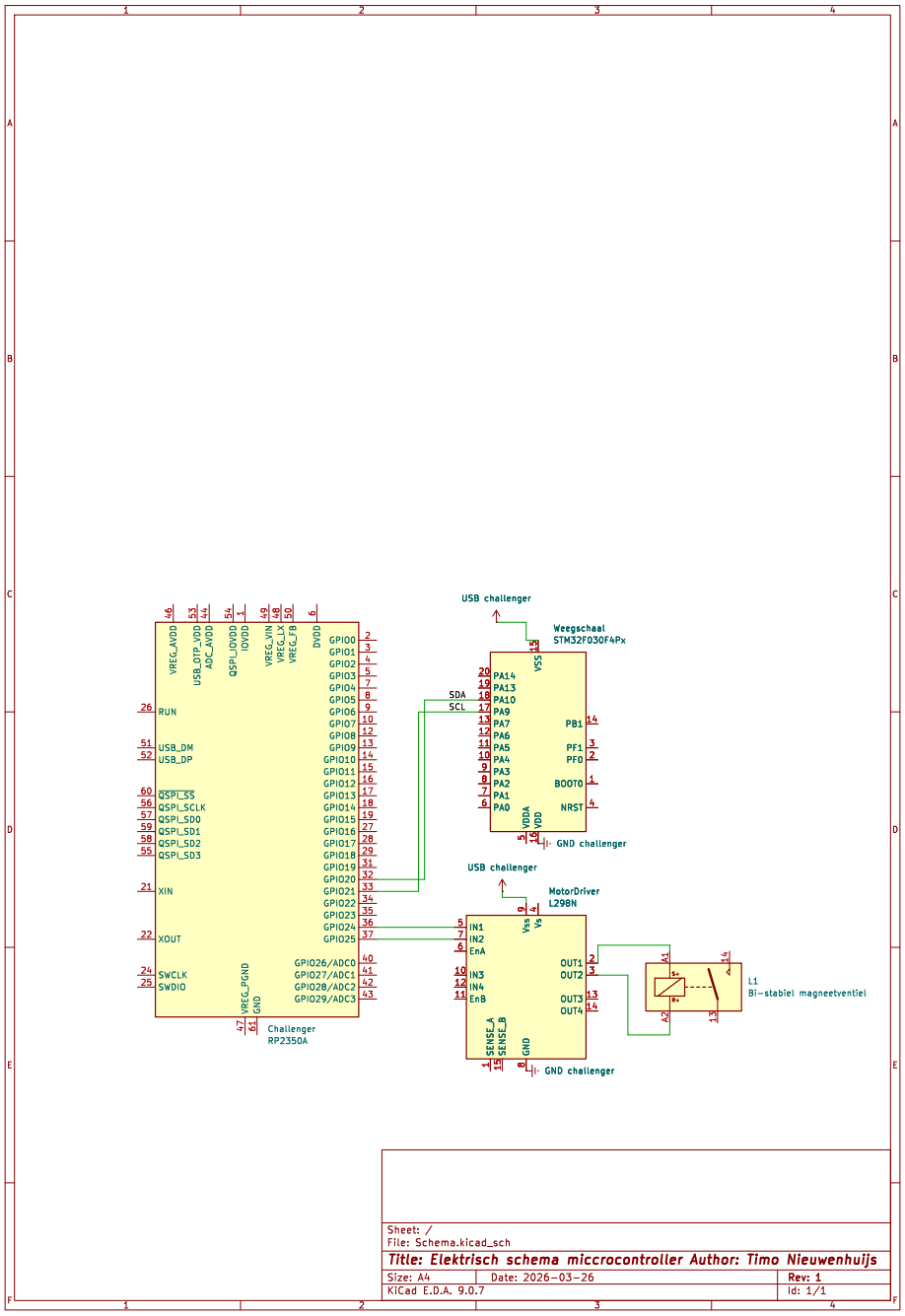
B.2 Microcontroller



Figuur 6: Flowchart van de microcontroller

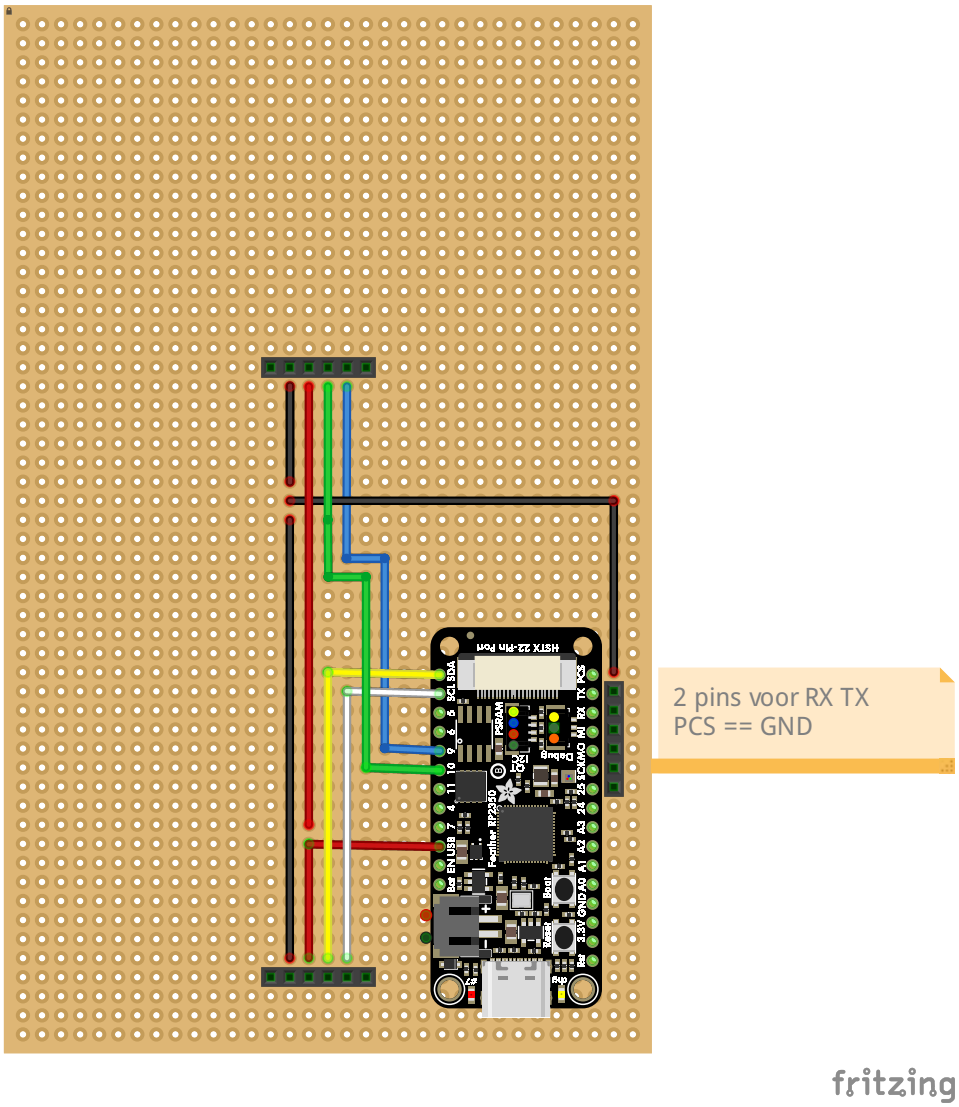
C Schema's

C.1 Elektrisch schema



Figuur 7: Elektrisch schema microcontroller

C.2 Soldeer schema



Figuur 8: Fritzing tekening microcontroller

D Broncode

D.1 Microcontroller

main.c

main.c > ...

```c
#include <stdio.h>
#include <string.h>

#include "pico/stdlib.h"
#include "hardware/i2c.h"
#include "MQTT.h"
#include "hardware/gpio.h"

#include "scale.h"
#include "motor.h"

#define MQTT_TOPIC_PUB  "tap/snsr"

#define PUB_INTERVAL_MS 50

// actuator commando ontvangen
void Use_data(const char *topic, const char *payload){
    if (strcmp(topic, "tap/actr") == 0) {
        printf("Ontvangen payload: %s\n", payload);

        if (strcmp(payload, "open") == 0) {
            open();
        }
        else if (strcmp(payload, "dicht") == 0) {
            dicht();
        }
    }
}

int main()
{
    setFeedBackRecieve(Use_data);

    stdio_init_all();
    mqtt_init();
    weigh_init();

    // solenoid valve dicht voor safety
    dicht();
    absolute_time_t volgende = make_timeout_time_ms(PUB_INTERVAL_MS);

    while (1)
    {
        SyncMqttData();

        // altijd gewicht checken
        weigh_loop();

        if (time_reached(volgende))
        {

            char payload[128];
            snprintf(payload, sizeof(payload),
                     "%.2f", gewicht);

            mqtt_send(payload, MQTT_TOPIC_PUB);

            volgende = make_timeout_time_ms(PUB_INTERVAL_MS);
        }
        sleep_ms(2);
    }
}
```

Figuur 9: Main programma

```c
motor.c > ...
#include <stdio.h>
#include "pico/stdlib.h"

// gpio pin nummers
#define IN1 25 // fysiek 9
#define IN2 24 // fysiek 6

void init() {
    // H-brug kanalen
    gpio_init(IN1);
    gpio_set_dir(IN1, GPIO_OUT);

    gpio_init(IN2);
    gpio_set_dir(IN2, GPIO_OUT);

    gpio_put(IN1, 0);
    gpio_put(IN2, 0);
}

void open() {
    init();
    gpio_put(IN1, 1);
    sleep_ms(20);
    gpio_put(IN1, 0);
}

void dicht() {
    init();
    gpio_put(IN2, 1);
    sleep_ms(20);
    gpio_put(IN2, 0);
}
```

(a) motor.c

```c
motor.h > ...
#ifndef _MOTOR_H_
#define _MOTOR_H_

void open();
void dicht();

#endif /* _MOTOR_H_ */
```

(b) motor.h

Figuur 10: Aansturing actuator

```c
scale.c > ...
#include <stdio.h>
#include <string.h>
#include "pico/stdlib.h"
#include "hardware/i2c.h"
#include "scale.h"

// I2C defines
#define I2C_PORT i2c0
#define I2C_SDA 20
#define I2C_SCL 21
#define SCALE_ADDR 0x26

// addresen binnen scale en data buffer
uint8_t zero = 0x50; // register voor offset
uint8_t reg = 0x10; // register van het gewicht als float in gram
uint8_t buffer[4]; // buffer van de data

// globale variabelen voor gewicht
float gewicht;

void weigh_init()
{
    // I2C Initialisation. Using it at 100Khz.
    i2c_init(I2C_PORT, 100*1000);

    // i2c pins
    gpio_set_function(I2C_SDA, GPIO_FUNC_I2C);
    gpio_set_function(I2C_SCL, GPIO_FUNC_I2C);
    gpio_pull_up(I2C_SDA);
    gpio_pull_up(I2C_SCL);

    //1x nullen
    i2c_write_blocking(i2c0, SCALE_ADDR, &zero, 1, false);
    i2c_write_blocking(i2c0, SCALE_ADDR, &reg, 1, false); // register die we
    i2c_read_blocking(i2c0, SCALE_ADDR, buffer, 4, false);

    sleep_ms(10);
}

void weigh_loop()
{
        // altijd wegen
        i2c_write_blocking(i2c0, SCALE_ADDR, &reg, 1, false); // register die
        i2c_read_blocking(i2c0, SCALE_ADDR, buffer, 4, false);  // check of a

        // copy naar andere variable voor verwerking
        memcpy(&gewicht, buffer, 4);
}
```

(a) scale.c

```c
main.c    scale.h
scale.h > ...
#ifndef _SCALE_H_
#define _SCALE_H_
void weigh_init();
void weigh_loop();

extern float gewicht;
#endif /* _SCALE_H_ */
```

(b) scale.h

Figuur 11: Uitlezing sensor

```c
// main.c
// RP2350 + ESP32-C6 (ESP-AT) hardcoded WiFi + MQTT demo)

#include <stdio.h>
#include <string.h>
#include <stdbool.h>

#include "pico/stdlib.h"
#include "MQTT.h"
#include "esp_at.h"
#include "esp_mqtt.h"
#include "esp_wifi.h"

#define WIFI_SSID       "TRG"
#define WIFI_WACHTWOORD "12345678"

#define MQTT_BROKER_IP  "10.42.0.1"
#define MQTT_POORT      1883
#define MQTT_CLIENT_ID  "tap"
#define MQTT_SCHEME     1   // 1 = MQTT over TCP (geen TLS)

#define MQTT_TOPIC_SUB  "tap/actr"
#define MQTT_TOPIC_PUB  "tap/snsr"

#define PUB_INTERVAL_MS 5000

mqtt_message_sent_function FeedBackFunctionSent = NULL;
mqtt_message_recieved_function FeedBackFunctionRecieve = NULL;

esp_mqtt_config_t mqtt_cfg = {
    .verbinding_id       = 0,                // Meestal 0, tenzij je meerdere verbindi
    .client_id           = MQTT_CLIENT_ID, // Unieke ID voor de broker
    .gebruikersnaam      = "",  // Je MQTT gebruikersnaam (of NULL)
    .wachtwoord          = "", // Je MQTT wachtwoord (of NULL)
    .keepalive_seconden  = 60,               // Standaard hartslag van de verbinding
    .schone_sessie       = true              // 'true' verwijdert oude data bij herstart
};

// ============================
// Bericht ontvangen
// ============================

bool Print_Output = true;

void setMQTToutput(bool Output){
    Print_Output = Output;
}

void setFeedBackSent(mqtt_message_sent_function function){
    FeedBackFunctionSent=function;
}

void setFeedBackRecieve(mqtt_message_recieved_function function){
    FeedBackFunctionRecieve=function;
}

void MQTT_get_message(const char *topic, const char *payload, int len, void *user) {
    if (Print_Output==true){
        printf("\r\n--- MQTT BERICHT ONTVANGEN ---\r\n");
        printf("Topic:   %s\r\n", topic);
        printf("Payload: %s\r\n", payload);
        printf("-----------------------------\r\n");
    }
    if (FeedBackFunctionRecieve){
        FeedBackFunctionRecieve(topic,payload);
    }
}

void MQTT_send_message(const char *topic, const char *payload) {
    if (Print_Output==true){
        printf("\r\n--- MQTT BERICHT VERZONDEN ---\r\n");
        printf("Topic:   %s\r\n", topic);
        printf("Payload: %s\r\n", payload);
        printf("-----------------------------\r\n");
    }
    if (FeedBackFunctionSent){
        FeedBackFunctionSent(topic,payload);
    }
}
```

(a) Deel 1

```c
// ============================
// Initialiseren
// ============================

bool mqtt_init(){
    DEBUG_PRINT("[INFO] RP2350 <-> ESP32-C6\r\n");

    // UART configuratie naar ESP
    esp_at_cfg_t configuratie = {
        .baudrate  = 115200,
        .tx_pin    = 4,
        .rx_pin    = 5,
        .reset_pin = 15
    };

    DEBUG_PRINT("[INFO] ESP-AT initialiseren\r\n");
    esp_at_init(&configuratie);

    //functie waar data heen moet van de topics waar je op ben gesubscibed
    esp_mqtt_set_bericht_callback(MQTT_get_message,"iets waarom is dit nodig (geen idee)


    DEBUG_PRINT("[INFO] Wachten op AT antwoord\r\n");
    if (!wacht_op_at(6000))
    {
        printf("[FOUT] Geen AT response. Controleer TX/RX pins en baudrate.\r\n");
        return false;
    }
    DEBUG_PRINT("[OK] ESP reageert op AT\r\n");

    (void)stuur_cmd_ok("Echo uitzetten (ATE0)", "ATE0\r\n", 1500);

    // ============================
    // WiFi
    // ============================
    DEBUG_PRINT("\r\n[INFO] WiFi verbinden (hardcoded AT)\r\n");

    if (!stuur_cmd_ok("WiFi modus station: AT+CWMODE=1", "AT+CWMODE=1\r\n", 3000)){
        printf("[FOUT] Geen Wifi response. Controleer TX/RX pins en baudrate.\r\n");
        return false;
    }

    DEBUG_PRINT("\r\n[INFO] Verbinden met netwerk\r\n");
    wifi_verbind(WIFI_SSID,WIFI_WACHTWOORD,20000);


    DEBUG_PRINT("\r\n[INFO] IP adres opvragen: AT+CIFSR\r\n");
    wifi_vraag_ip(6000);
    drain_uart(200);
    sleep_ms(1000);

    // ============================
    // MQTT
    // ============================
    DEBUG_PRINT("\r\n[INFO] MQTT verbinden (hardcoded AT)\r\n");

    esp_mqtt_configureer(&mqtt_cfg,6000);

    esp_mqtt_verbind(MQTT_BROKER_IP,MQTT_POORT,12000);

    esp_mqtt_subscribe(MQTT_TOPIC_SUB,0,8000);

    //esp_mqtt_subscribe("Makerspace/Motors",0,8000);

    DEBUG_PRINT("\r\n[INFO] MQTT status: AT+MQTTCONN?\r\n");
    esp_at_write("AT+MQTTCONN?\r\n");
    drain_uart(2000);

    DEBUG_PRINT("\r\n[INFO] MQTT subscribe lijst: AT+MQTTSUB?\r\n");
    esp_at_write("AT+MQTTSUB?\r\n");
    drain_uart(2000);
}

// ============================
// Bericht verzenden
// ============================

bool mqtt_send(char data[], char topic[]){
    bool status = esp_mqtt_publiceer(topic,data,1,true,15000);

    printf("STATUS: %d\n",status);

    if (status==0){
        DEBUG_PRINT("[INFO] MQTT PUB verzonden: %s\r\n", data);
        MQTT_send_message(topic, data);
        return true;
    }
    else {
        printf("[FOUT] MQTT PUBRAW mislukt\r\n");
        return false;
    }
```

(b) Deel 2

Figuur 12: MQTT communicatie deel 1 en 3

```
// ============================
// Syc
// ============================

void SyncMqttData(){
    esp_at_poll();
}
```

(a) Deel 3

```
#define DEBUG true

#if DEBUG
  #define DEBUG_PRINT(fmt, ...) printf("[DEBUG] " fmt, ##__VA_ARGS__)
#else
  #define DEBUG_PRINT(fmt, ...) // Doet helemaal niets
#endif

#ifndef _MQTT_MAIN_
#define _MGTT_MAIN_

void SyncMqttData();

bool mqtt_init();

bool mqtt_send(char data[], char topic[]);

void setMQTToutput(bool Output);

typedef void (*mqtt_message_sent_function)(
    const char *topic,
    const char *payload);

typedef void (*mqtt_message_recieved_function)(
    const char *topic,
    const char *payload);

void setFeedBackSent(mqtt_message_sent_function function);

void setFeedBackRecieve(mqtt_message_recieved_function function);


#endif
```

(b) Header

Figuur 13: Vervolg MQTT communicatie

D.2 Raspberry Pi 5

```c
Timo.c 2 ×
C > Users > Timo > Documents > #TI code repos > domotica > INLEVEREN > Pi_mqtt > Timo.c > ...
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdbool.h>
#include <stdlib.h>

#include "MQTT.h"
#include "Timo.h"
#include "sql_t.h"

// voor het toevoegen aan database
char *type = "gewicht";

// alle gewicht var
double gewicht = 0;          // live gewogen gewicht
double leeg_gewicht = 0;     // nul gewicht
double prev_gewicht = 0;     // live gewicht - 1 voor checken of t constant is
double start_gewicht = 0;    // begin gewicht zodra die opengaat
double log_gewicht = 0;      // gewicht dat naar de db wordt gezet oftewel verschil in start/live oftewel hoe

// bool states voor controle of kraan open of dicht is
bool kan_open = true;        // MAG de kraan open aka, staat er niks op?
bool is_open = false;        // of de kraan nu open is of niet
bool first = true;           // puur voor eerste message

// teller, om als pauze te gebruiken ipv sleep()
int i = 0;

// runt zodra message wordt ontvangen op de sub
void Message_Recieved_T(const char *topic, const char *payload){

    // als ontvangen topic dit is dan doe alles
    if (strcmp(topic, "tap/snsr") == 0) {
        // convert  payload naar double voor voorwaarden (ontvangt alleen maar gewicht in juiste vorm)
        gewicht = atof(payload);

        // checkt of het het eerste ontvangen bericht is voor nulpunt
        if (first == true) {
            leeg_gewicht = gewicht;
            first = false;
        }

        // kan kijken hiermee kijken of gewicht stabiel is
        prev_gewicht = gewicht;

        // meer gewicht dan leeg en of ie open mag en gewicht is 'stabiel'
        // +-0.3 is kleine marge want fluctueert heel klein beetje
        if (gewicht > leeg_gewicht + 10 && kan_open == true && (gewicht < prev_gewicht + 0.3 || gewicht > pre
            // verhoogt de teller totdat ie 100 is, dus gewicht moet voor bepaald aantal seconden stabiel zi
            // na 50ms * 1 gaat open (publish interval op challenger is 50ms) 40 == 2s
            i++;
            if (i == 40){
                MQTT_publish("tap/actr","open",0);
                kan_open = false;
                is_open = true;
                start_gewicht = gewicht;
            }
        }

        // zodra huidig gewicht groter is dan start gewicht + 200gram ga dicht en log en reset de waarden
        if (gewicht > start_gewicht + 200 && is_open == true){
            MQTT_publish("tap/actr","dicht",0);

            log_gewicht = gewicht - start_gewicht;
            insert_meting(log_gewicht, type);
            is_open = false;
            start_gewicht = 0;
            i = 0;
        }

        // als open is en gewicht komt onder start waarde aka glas weg, ga dicht, -2 is afwijking compensati
        if (is_open == true && gewicht < start_gewicht - 2) {
            MQTT_publish("tap/actr","dicht",0);

            log_gewicht = gewicht - start_gewicht;
            insert_meting(log_gewicht, type);
            is_open = false;
            start_gewicht = 0;
            i = 0;
        }

        // zorg dat alleen open kan als de weegschaal leeg is
        if (gewicht < leeg_gewicht + 10) {
            kan_open =  true;
        }

        // debug string
        // printf("gewicht:%.2f || leeg:%.2f || start:%.2f || prev:%.2f\n", gewicht, leeg_gewicht, start_gew
    }
}

void Timo_loop(){
```

(a) Voorwaarden

```c
void Timo_loop(){

}

bool Timo_Init(){
    // connect naar de database 1 maal bij opstarten
    db_connect();

    //Subscriben naar topics die je wilt ontvangen
    //LET OP!!!
    //iedereen ontvangt van alle topics dus filter goed!!!
    MQTT_subscribe("tap/snsr",0);

    //Callback instellen waar messages binnen komen
    if (set_callback(Message_Recieved_T)==false){
        return false;
    }

    return true;
}
```

(b) Initialisatie

Figuur 14: Dataverwerking op Pi

```c
C: > Users > Timo > Documents > #TI code repos > domotica > INLEVEREN > Pi_mqtt > C sql_t.c > ...
#include <stdio.h>
#include <stdlib.h>
#include <mariadb/mysql.h>
#include <string.h>

#include "sql_t.h"

MYSQL *conn_t;

void err_exit(char* s)
{
    perror(s);
    perror("\n");
    exit(1);
}

// moet worden aangeroepen ergens, 1x, dus best in de init.
int db_connect() {

    // Initialize Connection
   if (!(conn_t = mysql_init(0)))
   {
      fprintf(stderr, "unable to initialize connection struct\n");
      exit(1);
   }
   else
   {
    fprintf(stderr, "GOT it YOU ARE IN\n");
   }

   // Connect to the database
   if (!mysql_real_connect(
       conn_t,                  // Connection
       "localhost",             // Host
       "frap",                  // User account
       "1234",                  // User password
       "Watertap",              // Default database
       3306,                    // Port number
       NULL,                    // Path to socket file
       0                        // Additional options
     ))
   {
      // Report the failed-connection error & close the handle
      fprintf(stderr, "Error connecting to Server: %s\n", mysql_error(conn_t));
      mysql_close(conn_t);
      exit(1);
   }

    return 0;
}

// aanroepen als geen mqtt ontvangt, kan bij de alive check en disconnect bij geen puls van betreffende rp
int db_disconnect() {
    // Close the Connection
    mysql_close(conn_t);
    mysql_library_end();
    return 0;
}

int send_query(char *buffer){
    char * query;

    MYSQL_STMT *stmt = NULL;
    stmt = mysql_stmt_init(conn_t);

    // hier definier je de query - communicatie met database
    if (stmt) {
        query = buffer;

        // voorbereiding voor het uitvoeren van de query
        if (mysql_stmt_prepare(stmt, query, strlen(query))) {
            printf("Statement prepare failed: %s\n", mysql_stmt_error(stmt));
        } else {
            puts("Statement prepare OK!");
        }
    }

    // uitvoeren van de query
    if (mysql_stmt_execute(stmt)) {
            printf("Statement execute failed: %s\n", mysql_stmt_error(stmt));
        } else {
            puts("Statement execute OK!");
        }
    return 0;
}
```

(a) database verbinding

```c
// insert into meeting
int insert_meting(double meetwaarde, char *type) {
    // kan zub-zaar kaaya hier tabellen en kolommen hardcoden
    char *tabel = "meting";

    // buffer string voor de query, verwerkt ook invoer variables
    char buffer[256];
    snprintf(buffer, sizeof(buffer), "INSERT INTO %s (Meetwaarde, Meetdatum, sensorSensorId) VALUES ('%.2f', Now(), (SELECT SensorId FROM sensor where MetingType = '%s'));", tabel,  meetwaarde, type);

    //verstuurt het
    send_query(buffer);

    return 0;
}
```

(b) Insert query voor de meetwaarde

```c
#ifndef _SQL_T_
#define _SQL_T_

int db_connect();
int db_disconnect();

int insert_meting(double meetwaarde, char *type);

#endif
```

(c) Gebruikte header

Figuur 15: Database communicatie

```c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <mosquitto.h>
#include <stdbool.h>

#define PI_IP "0.0.0.0"

#include "MQTT.h"

struct mosquitto *mosq;


MQTT_CALLBACK Callback1 = NULL;
MQTT_CALLBACK Callback2 = NULL;
MQTT_CALLBACK Callback3 = NULL;

bool set_callback(MQTT_CALLBACK function){
    if (!Callback1) Callback1 = function;
    else if (!Callback2) Callback2 = function;
    else if (!Callback3) Callback3 = function;
    else return false;
    return true;
}

void on_message(struct mosquitto *mosq, void *obj, const struct mosquitto_mes
    char *topic = msg->topic;
    char *payload = (char *)msg->payload;

    (void)mosq;
    (void)obj;

    if (Callback1) Callback1(topic,payload);
    if (Callback2) Callback2(topic,payload);
    if (Callback3) Callback3(topic,payload);
}

void MQTT_subscribe(char topic[],int qos){
    mosquitto_subscribe(mosq, NULL, topic, qos);

}

void MQTT_publish(char topic[],char payload[], int qos){
    mosquitto_publish(mosq, NULL, topic, (int)strlen(payload), payload, qos,
}

int MQTT_init(){
    mosquitto_lib_init();

    mosq = mosquitto_new("teller_client2", false, NULL);
    if(!mosq) return 1;

    // Callback instellen voor inkomende berichten
    mosquitto_message_callback_set(mosq, on_message);

    if(mosquitto_connect(mosq, PI_IP, 1883, 60) != MOSQ_ERR_SUCCESS) {
        printf("Kan niet verbinden met broker.\n");
        return 1;
    }

    mosquitto_loop_start(mosq);

    return 0;

}
```

```c
#ifndef _MQTT_H_
#define _MQTT_H_

#include <stdbool.h>

int MQTT_init();

void MQTT_publish(char topic[],char payload[], int qos);

void MQTT_subscribe(char topic[],int qos);

typedef void (*MQTT_CALLBACK)(const char *topic, const char *payload);

bool set_callback(MQTT_CALLBACK function);


#endif
```

Figuur 16: MQTT communicatie

Makefile

```makefile
# --- 1. Compiler Settings ---
CC = gcc
CFLAGS = -std=c99 -Wall -Wextra -Wpedantic -Wconversion
LIBS = -lmosquitto -lmariadb
TARGET = main

# --- 2. Path Configuration ---
# Set these to 'yes' to include, 'no' to disable
USE_PARENT = yes
USE_PATH1 = yes
USE_PATH2 = yes
USE_PATH3 = yes

# Define the actual directory names
DIR1 = reamo
DIR2 = frap
DIR3 = silent-ghost

# --- 3. Logic to Collect Sources ---
SRCS = $(wildcard *.c)
INCLUDES = -I.

# Add Parent Folder sources
ifeq ($(USE_PARENT), yes)
    SRCS += $(wildcard ../*.c)
    INCLUDES += -I..
endif

# Add Subdirectory sources
ifeq ($(USE_PATH1), yes)
    SRCS += $(wildcard $(DIR1)/*.c)
    INCLUDES += -I$(DIR1)
endif

ifeq ($(USE_PATH2), yes)
    SRCS += $(wildcard $(DIR2)/*.c)
    INCLUDES += -I$(DIR2)
endif

ifeq ($(USE_PATH3), yes)
    SRCS += $(wildcard $(DIR3)/*.c)
    INCLUDES += -I$(DIR3)
endif

# --- 4. Object Generation ---
# Note: This handles the paths automatically
OBJS = $(SRCS:.c=.o)

.PHONY: all clean

all: $(TARGET)

# Link Step
$(TARGET): $(OBJS)
        $(CC) $(CFLAGS) -o $@ $(OBJS) $(LIBS)

# Compilation Step
# This rule tells make how to handle files regardless of which folder they are in
%.o: %.c
        $(CC) $(CFLAGS) $(INCLUDES) -c $< -o $@

clean:
        rm -f $(TARGET) $(OBJS)
        rm -f ../*.o $(DIR1)/*.o $(DIR2)/*.o $(DIR3)/*.o
        @echo "Cleaned all objects and binaries."
```

Figuur 17: Makefile voor programma's van alle projectleden